Pontifícia Universidade Católica do Rio Grande do Sul

Escola Politécnica

Sistemas Digitais | 98G02-04

Prof. Iaçanã Ianiski Weber

Laboratório

Implementação do RSA

6 de março de 2026

Objetivo Geral

Ao final destas sessões, você terá implementado os componentes centrais do criptossistema RSA em C++ utilizando a biblioteca GMP e terá usado sua implementação para decriptar uma imagem criptografada.

Pré-requisitos

- Conhecimento básico de programação em C++ (variáveis, loops, funções, entrada/-saída de arquivos).
- Um compilador C++ (como g++) e a biblioteca GMP instalada.
- Para compilar programas com GMP, tipicamente:

  ```
  g++ programa.cpp -o programa -lgmp -lgmpxx -std=c++11
  ```

  A *flag* `-lgmpxx` é para as classes C++ da GMP, `-lgmp` para a interface C.
- Familiaridade com operações básicas de linha de comando.

1 Números Grandes e Geração de Primos

Objetivo

Entender como trabalhar com números grandes utilizando a GMP e como gerar números primos grandes.

1.1 Introdução à GMP (GNU Multiple Precision Arithmetic Library)

Para construirmos algoritmos criptográficos como o RSA, precisaremos trabalhar com números inteiros gigantescos, que podem ter centenas ou até milhares de bits. Os tipos de dados padrão do C/C++ (`int`, `long long`, etc.) são muito pequenos para isso.

É aqui que entra a GMP (*GNU Multiple Precision Arithmetic Library*). A GMP é uma biblioteca especializada que nos permite realizar cálculos com "precisão arbitrária", ou seja, com números inteiros tão grandes quanto a memória do seu computador permitir. Sem ela, implementar o RSA de forma segura seria muito mais difícil.

O principal tipo de dado que usaremos da GMP para esses "super inteiros"é o `mpz_t`. Em nosso laboratório, vocês aprenderão o essencial sobre como:

- **Inicialização:**
  - `mpz_init(var)`
  - `mpz_init_set_ui(var, ulong)`
  - `mpz_init_set_str(var, "str", base)`
- **Atribuição:**
  - `mpz_set(rop, op)`
  - `mpz_set_ui(rop, ulong)`
  - `mpz_set_str(rop, "str", base)`
- **Aritmética:**
  - `mpz_add`
  - `mpz_sub`
  - `mpz_mul`
  - `mpz_fdiv_q`
- **Comparação:**
  - `mpz_cmp(op1, op2)`
  - `mpz_cmp_ui(op1, ulong)`
- **Exponenciação modular:**
  - `mpz_powm(rop, base, exp, mod)`
- **Máximo Divisor Comum (MDC):**
  - `mpz_gcd(rop, op1, op2)`
- **Inverso modular:**
  - `mpz_invert(rop, op1, op2)` calcula $op1^{-1} \pmod{op2}$
- **Números aleatórios:**
  - `gmp_randstate_t`
  - `gmp_randinit_default`
  - `gmp_randseed_ui`
  - `mpz_urandomb(rop, state, nbits)`
- **Saída:**
  - `gmp_printf("%Zd\n", var)`
- **Liberação de memória:**
  - `mpz_clear(var)`

1.2 Analisando o Código `primegen.cpp` Fornecido

Vamos analisar o código que foi fornecido:

```cpp
#include <iostream>
#include <time.h>
#include <gmp.h>
#include <stdio.h>

int main(int argc, char* argv[]){
    if(argc <= 1){
        std::cout << "Uso do primegen: \n > ./primegen <numero de bits> " << std::endl;
        return 1;
    }

    unsigned int size = atoi(argv[1]);

    mpz_t the_number;
    mpz_t the_number_minus_one;
    mpz_t the_two;
    mpz_t the_result;

    gmp_randstate_t state;
    gmp_randinit_default(state);
    gmp_randseed_ui(state, time(NULL));

    mpz_init_set_ui(the_two, 2);
    mpz_init(the_number);
    mpz_init(the_number_minus_one);
    mpz_init(the_result);

    std::cout << "Metodo de Fermat - buscando numero primo de " << size << " bits." << std::endl;

    int flag = 0, tentativa = 0;
    do{
        tentativa++;
        std::cout << "\rTentativa: " << tentativa << std::flush;

        mpz_urandomb(the_number, state, size);

        mpz_setbit(the_number, 0);
        if (size > 1) {
             mpz_setbit(the_number, size - 1);
        }

        mpz_sub_ui(the_number_minus_one, the_number, 1);
        mpz_powm(the_result, the_two, the_number_minus_one, the_number);

        flag = mpz_cmp_ui(the_result, 1);

    } while(flag != 0);

    gmp_printf("\nPrimo encontrado: %Zd\n", the_number);

    mpz_clear(the_two);
    mpz_clear(the_number);
    mpz_clear(the_number_minus_one);
    mpz_clear(the_result);
    gmp_randclear(state);

    return 0;
}
```

Pontos de Discussão sobre o `primegen.cpp`:

1. **Propósito:** Gerar um número provavelmente primo com um tamanho de bits especificado.
2. **Teste de Primalidade:** Utiliza o **Pequeno Teorema de Fermat**: se N é primo, então para qualquer inteiro a não divisível por N, $a^{N-1} \equiv 1 \pmod{N}$. O código usa $a = 2$.
3. **Natureza Probabilística:**
   - **Importante:** Este é um teste probabilístico. Se $2^{N-1} \equiv 1 \pmod{N}$, N é *provavelmente* primo.
   - **Números de Carmichael:** Existem números compostos N para os quais $a^{N-1} \equiv 1 \pmod{N}$ para todos os inteiros a que são coprimos com N. Portanto, este teste não é infalível.
   - Para testes de primalidade mais robustos, o teste de Miller-Rabin é comumente usado.
4. **Geração de Números Aleatórios:** `mpz_urandomb` gera um número aleatório.

Exercício 1: Gerando Dois Primos (p e q)

- **Tarefa:** Crie um novo arquivo C++ (ex: `generate_pq.cpp`).
- Adapte a lógica do `primegen.cpp` para gerar dois números primos grandes *distintos*, p e q.
- Garanta que $p \neq q$.
- Imprima p e q no console usando `gmp_printf`.
- Compile e execute seu programa.

---

2 Geração de Chaves RSA (Conceito Chave)

Objetivo

Implementar o algoritmo de geração de chaves RSA usando os primos p e q.

2.1 Passos para Geração de Chaves RSA (Recapitulando)

1. Escolha dois números primos grandes distintos, p e q.
2. Calcule $n = p \cdot q$. Este é o **módulo (modulus)**.
   - `mpz_mul(n, p, q);`

3. Calcule a função Phi de Euler: $\phi(n) = (p-1)(q-1)$.
   - `mpz_sub_ui(p_m1, p, 1);`
   - `mpz_sub_ui(q_m1, q, 1);`
   - `mpz_mul(phi_n, p_m1, q_m1);`
4. Escolha um inteiro e (o **expoente público**) tal que $1 < e < \phi(n)$ e $\gcd(e, \phi(n)) = 1$.
   - Uma escolha comum para e é 65537.
   - `mpz_gcd(gcd_val, e, phi_n);`
   - Verifique se `mpz_cmp_ui(gcd_val, 1) == 0`.
5. Calcule d (o **expoente privado**) tal que $d \cdot e \equiv 1 \pmod{\phi(n)}$. ($d$ é o inverso multiplicativo modular de e módulo $\phi(n)$).
   - `mpz_invert(d, e, phi_n);`.

Chave Pública (Public Key): (n, e)
Chave Privada (Private Key): (n, d)

Exercício 2: `rsakeygen.cpp`

- **Tarefa:** Crie um programa C++ `rsakeygen.cpp`.
- Este programa deve:
  1. Receber um tamanho de bits como argumento de linha de comando.
  2. Gerar p e q.
  3. Calcular n e $\phi(n)$.
  4. Definir e (ex: 65537) e verificar $\gcd(e, \phi(n)) = 1$.
  5. Calcular d.
  6. Exibir p, q, n, e, d.
- Lembre-se de inicializar `mpz_init` e limpar `mpz_clear` as variáveis.
- Use `mpz_get_str` para converter `mpz_t` para string.
- **Saída:** Exiba o resultado no console ou em arquivos: `public_key.txt`, `private_key.txt`

---

3 Funções de Criptografia e Decriptografia

Objetivo

Implementar as operações matemáticas de criptografia e decriptografia RSA.

3.1 Operações RSA

Seja M a mensagem em texto claro (plaintext), $0 \leq M < n$.

- **Criptografia (Encryption):** $C = M^e \pmod{n}$
  - `mpz_powm(C, M, e, n);`
- **Decriptografia (Decryption):** $M = C^d \pmod{n}$
  - `mpz_powm(M, C, d, n);`

Exercício 3: Implementando funções `encrypt` e `decrypt`

- **Tarefa:** Crie duas funções C++ (ex: em `rsa_test.cpp`).

```cpp
// Funcao para criptografar uma mensagem M
void rsa_encrypt(mpz_t C, const mpz_t M, const mpz_t e, const mpz_t n) {
    mpz_powm(C, M, e, n);
}

// Funcao para decriptar um texto cifrado C
void rsa_decrypt(mpz_t M_decrypted, const mpz_t C, const mpz_t d, const
    mpz_t n) {
    mpz_powm(M_decrypted, C, d, n);
}
```

- No seu `main` para `rsa_test.cpp`:
  1. Inicialize n, e, d e uma mensagem M (ex: `mpz_init_set_ui(M, 12345UL);`).
  2. Chame `rsa_encrypt` para obter C. Imprima C.
  3. Chame `rsa_decrypt` em C para obter M'. Imprima M'.
  4. Verifique se $M' = M$.
  5. Exemplo de saída (textos aqui podem ter acentos, pois é saída de console, não código):

     ```
     Mensagem Original: 12345
     Texto Cifrado (Ciphertext): <algum numero grande>
     Mensagem Decriptada: 12345
     ```

---

4 Aplicação em Criptografia/Decriptografia de Texto

Objetivo

Aplicar sua implementação RSA para criptografar e decriptar uma string de texto curta.

4.1 Convertendo Texto para Números para o RSA

O RSA opera sobre números.

- **Abordagem simples (Caractere por Caractere):**
  - Tratar cada caractere como seu valor inteiro ASCII.
  - Criptografar/decriptar este inteiro. Requer $n > 255$.
  - Exemplo: "OI"→ O (ASCII 79), I (ASCII 73). Criptografa 79, depois 73.
- **Para este exercício, usaremos criptografia caractere por caractere.**

Exercício 4: `rsatext.cpp`

- **Tarefa:** Crie um programa C++ `rsatext.cpp`.
- Este programa deve:
  1. **Carregar Chaves:** Ler n, e, d de arquivos.
  2. **Entrada (Input):** Receber uma string curta (`std::getline`).
  3. **Criptografia (Encryption):**
     - Para cada caractere, converta para `mpz_t` (valor ASCII).
     - Criptografe com `rsa_encrypt`.
     - Imprima cada número criptografado (use `mpz_get_str` e `free`).
  4. **Decriptografia (Decryption):**
     - Para cada número criptografado (string), converta para `mpz_t`.
     - Decripte com `rsa_decrypt`.
     - Converta o `mpz_t` decriptado para `unsigned long` (`mpz_get_ui`), depois para `char`.
     - Monte a string original.
  5. **Saída (Output):** Imprimir string original, números criptografados, string recuperada.

Boa sorte e divirta-se construindo o RSA!